FORMULA MANAGEMENT

5-Module Governance Platform for Formula-Driven Project Workflows

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 · 2026 · Yohannes Bayeh

Field	Detail
Project Name	Formula Management
Type	React-Only Single Page Application (SPA) — Private Enterprise Project
Developer	Yohannes Bayeh
Year	2026
Primary UI Color	Emerald Green — #10B981
Stack	React 19 (Vite) · Tailwind CSS · React Context API · Browser localStorage
Deployment	GitHub Pages (static build)
Access	Private project — no public demo or repository links included

SECTION 1 — EXECUTIVE SUMMARY, PROBLEM STATEMENT & SOLUTION

1.1 Executive Summary

Formula Management is a pure React Single Page Application (SPA) that delivers a 5-module governance platform purpose-built for formula-driven project workflows. It replaces manual, spreadsheet-based resource allocation and unmonitored budgeting with a structured, rule-based system for allocation, compliance monitoring, scheduling, proposal governance, and feasibility evaluation.

The application is architected as a fully client-side system — there is no backend server or database. All state is managed through React's Context API and persisted in the browser via localStorage, making the platform lightweight, portable, and simple to deploy as a static build. The UI is built around a dark navy/slate visual theme accented with Emerald Green, reflecting the platform's compliance-driven identity, where green signals safe, within-threshold status.

1.2 Problem Statement

Problem	Impact
Resource allocation tracked manually in spreadsheets	Error-prone, version-conflicted, difficult to reconcile
No systematic threshold/boundary enforcement	Budget or resource overruns go unnoticed until it's too late
No structured formula change history	Impossible to trace who changed an allocation rule, or when
Scheduling and resource conflicts handled reactively	Conflicts discovered after they already impact a project
Proposal and feasibility review handled informally	Inconsistent evaluation standards, no enforced approval stages
No unified governance view across modules	Fragmented oversight across allocation, compliance, and scheduling

1.3 Solution

- A configurable Formula Engine with immutable, parent-linked version history for every allocation rule change.
- A Corridor Engine that continuously monitors allocations against defined min/max/threshold boundaries, with real-time visual compliance indicators.

- A Schedule Matrix offering Gantt-based visualization, resource conflict detection, and forecasting informed by 4,000+ past projects.
- A Concept Administration workflow enforcing a structured, 3-stage proposal approval chain.
- A Feasibility Administration workflow for structured study registration, evaluator assignment, and standardized scoring.
- A strict Role-Based Access Control (RBAC) model governing what each user type can view and act on within the interface.

SECTION 2 — CORE FEATURES & 5-MODULE BREAKDOWN

2.1 The 5 Core Functional Modules

Module	Description	Key Capability
Module 1 — Formula Engine	Configurable, rule-based resource allocation engine	Immutable version history, parent-child version tracking, override workflows
Module 2 — Corridor Engine	Min/max/threshold boundary compliance monitoring	Real-time GREEN (safe compliance) / RED (threshold exceeded) visual indicators
Module 3 — Schedule Matrix	Gantt chart visualization and resource planning	Resource conflict detection; forecasting from 4,000+ historical projects
Module 4 — Concept Administration	Digital proposal submission and review	3-stage approval chain: Reviewer → Committee → Management
Module 5 — Feasibility Administration	Structured feasibility study management	Study registration, evaluator assignment, standardized scoring matrices

2.2 Supporting Features

Feature	Description
Interactive Tabbed Navigation	Each module is accessible via a persistent tab interface for fast context-switching
Data-Dense Sortable & Filterable Tables	Built using React Table patterns for allocation records, schedules, proposals, and studies
One-Click CSV Export	Approved allocation reports can be exported to CSV directly from the client
Strict Client-Side Immutable Versioning	Editing an approved formula creates a new version linked via parentId, rather than overwriting it

SECTION 3 — TECHNICAL ARCHITECTURE & DESIGN SYSTEM

3.1 Architecture Overview

Formula Management is a pure frontend application with no server-side component. All business logic — formula evaluation, corridor compliance checks, versioning rules, and workflow state transitions — runs entirely in the browser.

```
[React 19 SPA -- built with Vite]         |         |-- Tailwind CSS (Emerald Green
design system)         |-- React Context API (global application state)         |--
Browser localStorage (persistent client-side data store)         |         |-- Formula
Engine (versioning, override logic)         |-- Corridor Engine (threshold evaluation,
compliance status)         |-- Schedule Matrix (Gantt rendering, conflict detection)
|-- Concept Administration (approval workflow state machine)         \-- Feasibility
Administration (scoring & evaluator logic)
```

Because there is no backend server or database, all persistence is handled client-side via localStorage, and the entire application is distributed as a static build.

3.2 Technology Stack

Layer	Technology	Purpose
Framework	React 19	Component-based UI with hooks
Build Tool	Vite	Fast development server and optimized static production build
Styling	Tailwind CSS	Utility-first styling system implementing the Emerald Green design language
State Management	React Context API	Global state sharing across modules without external state libraries
Data Persistence	Browser localStorage	Client-side storage for formulas, versions, schedules, and proposals
Table Rendering	React Table patterns	Sortable, filterable data-dense tables across all modules
Deployment	GitHub Pages	Static hosting of the production Vite build
Backend	None	Pure client-side SPA — zero server or database dependency

3.3 Design System — Emerald Green Branding

Element	Value	Usage
Background	#07090F (Dark Navy / Near-Black)	Page background
Card Container	#0D1117	Module panels, tables, and content cards
Primary Accent	#10B981 (Emerald Green)	Action buttons, active tab indicators, safe compliance status
Compliance — Safe	#10B981 (Emerald Green)	Corridor Engine “GREEN” status — within threshold
Compliance — Alert	Red (alert tone)	Corridor Engine “RED” status — threshold exceeded
Text Primary	White / near-white	Headings and primary labels
Text Muted	Muted gray	Secondary text, metadata, timestamps

3.4 UI Layout Principles

- Dark, low-glare interface suited for data-dense, long-session governance work.
- Emerald Green reserved specifically for affirmative/safe states and primary actions, reinforcing the Corridor Engine's compliance signaling.
- Tabbed module structure keeps all 5 modules accessible without deep navigation.
- Tables prioritize density and scannability, supporting sort/filter operations for large historical datasets (e.g., the 4,000+ project forecasting base).

SECTION 4 — FORMULA VERSIONING LOGIC & CORRIDOR COMPLIANCE

4.1 Immutable Formula Versioning

Formula Management enforces a strict, client-side immutable versioning model for all allocation formulas:

Rule	Behavior
Approved formulas are immutable	Once approved, a formula's rule definition cannot be edited in place
Edits create new versions	Modifying an approved formula generates a new version record rather than overwriting the original
Parent-child linkage	Each new version stores a parentId reference to the version it was derived from, preserving full lineage
Override workflow	Changes to an approved formula are routed through an override workflow rather than a direct edit
Single active version	Only one version of a given formula is considered “active” at any time

4.2 Versioning Lifecycle

Stage	Behavior
Draft	Formula is editable prior to approval
Approved	Formula becomes immutable; no in-place edits permitted
Override Requested	A new version is created, linked to the prior version via parentId
New Version Approved	The new version becomes active; the prior version is retained for history/audit purposes

4.3 Corridor Compliance Rules

The Corridor Engine continuously evaluates active formulas and allocations against defined boundaries:

Boundary Type	Description
Minimum Threshold	The lowest acceptable allocation value before a compliance flag is raised
Maximum Threshold	The highest acceptable allocation value before a compliance flag is raised
Safe Corridor (GREEN)	Allocation falls within the defined min/max range — indicated in Emerald Green
Violation (RED)	Allocation falls outside the defined boundary — indicated in red, signaling an alert condition

Compliance status is calculated and displayed in real time within the client as allocation data changes, giving users immediate visual feedback without requiring a page reload or server round-trip.

SECTION 5 — ROLE-BASED VIEWS & FUNCTIONAL REQUIREMENTS

5.1 Role-Based Access Control (RBAC) Views

Formula Management implements five distinct role-based views within the client application:

Role	Scope
System Admin	Full configuration access across all modules and settings
Formula Admin	Creates and manages formulas; initiates override workflows for approved versions
Evaluator	Assigned to feasibility studies and proposal reviews; submits scoring and assessments
Viewer	Read-only access to dashboards, corridor status, and reporting views
Integration Service	Scoped view representing automated or system-level data interactions

Note: As a pure client-side SPA with no backend, role views control what is rendered and actionable within the interface. There is no server-side authorization layer, since there is no server component in this architecture.

5.2 Functional Requirements

ID	Requirement	Priority	Module
FR-01	Create and configure resource allocation formulas	High	Formula Engine
FR-02	Enforce immutable versioning with parentId lineage	High	Formula Engine
FR-03	Evaluate allocations against min/max/threshold boundaries	High	Corridor Engine
FR-04	Display real-time GREEN/RED compliance indicators	High	Corridor Engine
FR-05	Render Gantt chart schedule visualization	High	Schedule Matrix

ID	Requirement	Priority	Module
FR-06	Detect resource conflicts across scheduled items	Medium	Schedule Matrix
FR-07	Forecast scheduling outcomes from historical project data	Medium	Schedule Matrix
FR-08	Submit digital proposals through a 3-stage approval chain	High	Concept Administration
FR-09	Register feasibility studies and assign evaluators	High	Feasibility Administration
FR-10	Apply standardized scoring matrices to feasibility studies	Medium	Feasibility Administration
FR-11	Export approved allocation reports to CSV	Medium	Platform-wide
FR-12	Persist all application state in browser localStorage	High	Platform-wide
FR-13	Present role-specific views per RBAC role	High	Platform-wide

5.3 Non-Functional Requirements

ID	Requirement	Target
NFR-01	Client-side compliance calculation response time	Instant — no server round-trip
NFR-02	No backend or database dependency	Pure frontend SPA
NFR-03	Data persistence across sessions	Retained via localStorage until browser storage is cleared
NFR-04	Audit trail retention	Non-editable, append-only version and change history
NFR-05	Responsive layout	Usable across desktop and tablet viewport sizes

SECTION 6 — TESTING, LIMITATIONS & DEPLOYMENT

6.1 Deployment Configuration

Component	Platform	Detail
Application	GitHub Pages	Static hosting of the production Vite build
Build Command	npm run build	Vite production build output to dist/
Backend	None	No server required — pure static SPA deployment
Database	None	No database — all persistence handled via localStorage

6.2 Known Limitations (By Design)

Limitation	Description
No server-side enforcement	RBAC views control the interface only; there is no backend authorization layer
No cross-device sync	Data persisted in localStorage is local to a single browser/device
No multi-user collaboration	Each browser instance maintains its own independent state
Data loss on storage clear	Clearing browser storage removes all formulas, versions, and history
No true non-editable audit log	Audit retention logic is enforced at the application level, not by an immutable server-side store

These constraints are intentional for the current release, which is scoped as a private, client-side enterprise demonstration of the governance workflows rather than a multi-user production deployment.

6.3 Testing Considerations

Test Area	Approach
Formula Versioning	Verify editing an approved formula creates a new version with correct parentId linkage, rather than overwriting the original
Corridor Compliance	Verify GREEN/RED status updates correctly and immediately as allocation values change relative to defined thresholds
Approval Workflow	Verify proposals move correctly through Reviewer → Committee → Management stages
Data Persistence	Verify formulas, versions, and workflow state persist correctly across browser reloads via localStorage
CSV Export	Verify exported CSV files accurately reflect approved allocation data
Role-Based Views	Verify each of the 5 roles renders the correct scoped view and available actions

6.4 Recommended Future Enhancements

- Introduce an optional backend layer to enable true multi-user collaboration and server-enforced RBAC.
- Add cross-device data sync via an external persistence service.
- Strengthen audit trail immutability with a tamper-evident, append-only storage mechanism.
- Expand historical forecasting data beyond the current 4,000+ project baseline.

APPENDIX — DOCUMENT CONTROL

Version	Date	Author	Changes
1.0	July 27, 2026	Yohannes Bayeh	Initial documentation — complete